



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение высшего
образования
«МИРЭА Российский технологический университет»
РТУ МИРЭА

**Оптимизация культурооборота на рассадно-овощном
тепличном комбинате**

Выполнил студент 2 курса группы ЭФБО-04-24
Валекжанин Владимир Сергеевич

г. Москва
2026 год

Содержание

Содержание.....	2
Введение.....	4
1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ.....	6
1.1 Анализ предметной области.....	6
1.1.1 Предметная область.....	6
1.1.2 Анализ информационных процессов.....	6
1.1.3 Проблематика и пути к ее разрешению.....	7
1.2 Анализ аналогов разрабатываемого решения.....	8
1.3 Описание требований к разрабатываемому решению.....	8
2 ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ.....	11
2.1 Выбор инструментов или средств реализации проекта.....	11
2.1.1 Сводная таблица выбранных инструментов.....	11
2.1.1 Выбор СУБД.....	12
2.1.2 Выбор пакетного менеджера.....	13
2.1.3 Назначение библиотек тестирования.....	13
2.1.4 Выбор ORM-системы.....	14
2.1.5 Выбор хостинга кода.....	15
2.1.6 Выбор HTTP-обработчика.....	15
2.2 Разработка проекта решения.....	15
2.2.1 Упрощенная система ролей.....	15
2.2.1 Диаграмма сценариев использования.....	16
2.2.2 Entity-Relationship диаграмма (ERD).....	17
2.2.3 Диаграмма последовательности.....	18
3 ПРАКТИЧЕСКАЯ ЧАСТЬ.....	19
3.1 Разработка составных элементов.....	19
3.2 Установка и настройка.....	19
3.3 Тестирование.....	19
3.4 Ввод в эксплуатацию.....	19
3.5 Разработка сопроводительной документации.....	19
Заключение.....	20
Список использованных источников.....	21
Приложения.....	22
Приложение А: техническое задание продукта.....	22
А.1 Общие сведения.....	22
А.2 Назначение и цели создания системы.....	22

A.2.1 Назначение системы.....	22
A.2.2 Цели создания системы.....	23
A.3 Требования к системе.....	24
A.3.1 Требования к источникам данных системы.....	24
A.3.2 Требования к режимам функционирования системы.....	25
A.3.3 Требования к эргономике и технической эстетике.....	25

Введение

У агрономов защищенного грунта широкий круг обязанностей: от разработки технологических карт и контроля фитосанитарного состояния до учета ресурсов и финансовых затрат. Промышленное овощеводство — это высокотехнологичный процесс, где ошибка в планировании сроков на 5 дней может привести к потере до 20% урожайности. Мы решили создать систему, которая автоматизирует расчет потребностей и оптимизирует графики работ, отвечая на главный вопрос бизнеса: «Как максимально эффективно использовать каждый метр теплицы, чтобы получить прибыль?»

Данная работа актуальна, так как в сфере российской аграрной промышленности нет аналогичных продуктов. Наши конкуренты либо не имеют такой функциональности, либо она реализована частично.

Современный рынок разработки программного обеспечения обязывает разработчиков программного обеспечения не только быть специалистами своей области, но и ориентироваться в большинстве смежных технологических областей. Данная работа посвящена разработке современного программного обеспечения в соответствии с поставленной задачей и темой работы. Такая деятельность, помимо формирования практического опыта, позволяет сформировать новое видение классических задач по разработке программных комплексов и программного обеспечения.

Наша аудитория — агрономы и директора рассадно-овощных тепличных комбинатов, которые хотят ускорить конвейер культурооборота без ущерба качеству.

Цель работы заключается в разработке ПО для автоматизированного администрирования и оптимизации производственных циклов в условиях промышленного культурооборота.

Объектом данного исследования является промышленный культурооборот в условиях тепличного комбината.

Предметом исследования является цифровое моделирование производственных циклов защищенного грунта для обоснованного распределения площадей в условиях ограниченных ресурсов предприятия.

В качестве основных **методов исследования** применены анализ, синтез, сравнение и моделирование. Практическая реализация поставленной задачи соответствует основным подходам к разработке программного обеспечения.

Информационная база исследования содержит онлайн-статьи независимых авторов, справочники федерального назначения по ведению аграрного хозяйства.

Задачи работы:

- Провести анализ предметной области;
- проанализировать аналоги разрабатываемого решения;
- описать требования к разрабатываемому решению;
- выбрать инструменты и средства для реализации проекта;
- разработать проект решения;
- разработать программный продукт;
- составить отчет по работе;
- представить отчет к защите и ответить на вопросы по проекту.

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Анализ предметной области

1.1.1 Предметная область

Рассадно-овощной тепличный комбинат представляет собой сельскохозяйственное предприятие, где овощи и посадочный материал выращиваются в контролируемых условиях. Главная особенность таких хозяйств заключается в постоянном использовании площадей. Земля в теплицах работает без длительных перерывов, поэтому специалисты выстраивают четкий график смены культур. Этот график называют культурооборотом. Он позволяет получать урожай круглый год и поставлять продукцию в магазины без существенных пауз.

Основу работы составляет технологическая карта. В ней прописаны все шаги выращивания конкретного гибрида: от подготовки грунта до сбора урожая. Агрономы опираются на эти карты при составлении общего плана. План должен учитывать биологические особенности растений, сроки их созревания и требования к условиям среды. Особое внимание уделяется чередованию культур. Некоторые растения нельзя высаживать друг за другом на одном и том же месте, иначе возрастает риск болезней и снижается урожайность. Поэтому планирование превращается в задачу, где нужно совместить множество условий одновременно.

1.1.2 Анализ информационных процессов

Управление таким хозяйством требует постоянного сбора и обработки данных. Ежедневно агрономы сталкиваются с большими объемами информации. Необходимо отслеживать состояние каждой теплицы, фиксировать даты посева и высадки, контролировать расход материалов и трудозатрат. Все эти сведения формируют основу для принятия решений о дальнейших посадках.

В большинстве случаев работа ведется с помощью электронных таблиц и бумажных журналов. Данные разрозненны, и их обновление требует значительного времени. При составлении нового графика специалист вручную проверяет доступность площадей, сверяет сроки вегетации и вспоминает правила совместимости культур. Такая система хранения информации создает риск потери важных деталей. Ошибка в расчетах или пропущенное ограничение могут привести к наложению циклов, когда двум культурам требуется один и тот же блок одновременно. Кроме того, исторические данные о предыдущих посадках часто остаются без внимания, что усложняет соблюдение правил чередования. Информация существует, но она не связана в единую рабочую среду, что замедляет процесс планирования.

1.1.3 Проблематика и пути к ее разрешению

Ручное планирование культурооборота сталкивается с несколькими серьезными трудностями. Первая из них связана с человеческим фактором. При работе с большим количеством гибридов и блоков легко допустить ошибку в датах или забыть учесть требования предшественников. Вторая проблема заключается в неэффективном использовании площадей. Без точных расчетов теплицы могут простаивать между циклами, что снижает общую рентабельность производства. Третья сложность состоит в отсутствии наглядного инструмента контроля. Трудно представить загруженность всего комбината на год вперед, используя только списки и таблицы.

Внедрение специализированного программного обеспечения позволяет устранить эти трудности. Система берет на себя проверку совместимости культур и контроль временных окон. Алгоритмы автоматически распределяют посадки по свободным площадям, исключая конфликты и простои. Все данные хранятся в едином хранилище, что упрощает доступ к истории циклов и нормативам. Визуализация графика в виде временной шкалы дает агроному четкое представление о загрузке теплиц.

Таким образом, ключевая проблема заключается в отсутствии инструмента, который объединяет нормативные данные, алгоритмы оптимизации и наглядное планирование в единую рабочую среду. Разработка такой системы позволит сократить время на составление графиков, повысить точность расчетов и обеспечить стабильную работу конвейера продукции.

1.2 Анализ аналогов разрабатываемого решения

Ниже приведена сводная таблица 1.1, в которой мы рассмотрели нескольких конкурентов нашего решения. Далее мы отдельно рассмотрели каждого из конкурентов и сделали выводы о конкурентоспособности нашего решения.

Таблица 1.1 — сравнительный анализ конкурентов по наличию тех или иных функций

Конкурент	Управление ТК	Советы по оптимизации производства	Визуальное расположение объектов на карте	Исторические сведения с учетом предшественников
https://onesoil.ai	Нет	Частично	Да	Нет
https://agrobaserp	Нет	Нет	Нет	Нет
https://agro-scout.ru	Нет	Нет	Да	Нет
https://agrosignal.com	Нет	Нет	Частично	Нет
https://agrooffice.ru	Частично	Нет	Нет	Частично
agrar.io	Да	Да	Да	Да

Рассмотрим каждого из конкурентов подробнее:

- Сервис onesoil.ai работает со спутниковыми данными и создает карты полей с вегетационными индексами. Платформа визуализирует объекты на карте и дает частичные рекомендации по оптимизации. Фокус системы направлен на мониторинг текущего состояния посевов. Управление технологическими картами и учет предшественников не входят в функционал сервиса, поэтому

предварительное планирование циклов выращивания остается за пределами возможностей платформы;

- agrobase функционирует как справочник средств защиты растений и вредителей. Приложение содержит каталоги пестицидов с описаниями и рекомендациями по применению. Инструменты для работы с технологическими картами, визуализации площадей и оптимизации производства не предусмотрены. Сервис решает узкую задачу информационного сопровождения, не затрагивая вопросы планирования производственных циклов;
- agro-scout использует дроны и датчики для мониторинга полей. Система отображает объекты на карте и собирает данные о состоянии посевов в реальном времени. Платформа ориентирована на скаутинг и контроль уже растущих культур. Функции управления технологическими картами и советы по оптимизации производства отсутствуют, что ограничивает применение системы задачами наблюдения без инструментов предварительного планирования;
- agroSignal специализируется на мониторинге сельхозтехники и контроле полевых работ. Платформа отслеживает местоположение тракторов, расход топлива и выполнение операций в реальном времени. Система отображает технику на карте и собирает данные с GPS-трекеров. Функционал для работы с технологическими картами и планирования севооборота отсутствует. AgroSignal решает задачи оперативного контроля, но не помогает агроному заранее распределить культуры по площадям с учетом биологических требований;
- agroOffice представляет собой систему автоматизации учета для РОТК. Программа ведет складской учет, рассчитывает потребность в семенах и удобрениях, формирует отчетность. Здесь присутствует частичное управление технологическими картами в виде учета выполненных работ. Программа запоминает историю операций на полях, что позволяет частично отслеживать предшественников. При этом инструменты оптимизации размещения культур

и визуализация на карте не предусмотрены. AgroOffice фокусируется на документообороте и складском учете, оставляя вопросы стратегического планирования культурооборота без автоматизации.

Итого, наша система заполняет пробелы, которые оставляют рассмотренные конкуренты. Мы объединяем математическую оптимизацию размещения культур, управление технологическими картами, визуализацию на карте и строгий учет предшественников в одном решении. Алгоритм автоматически распределит посадки по свободным площадям, исключая конфликты и простои, а диаграмма Ганта даст наглядное представление о загрузке теплиц на весь сезон. Такой подход устраняет необходимость использования нескольких разрозненных инструментов.

1.3 Описание требований к разрабатываемому решению

Для того чтобы система действительно помогала агрономам в их ежедневной работе, необходимо четко понимать, каким критериям она должна соответствовать. Полное техническое задание с детальным описанием всех параметров приведено в Приложении А.

В первую очередь система должна решать конкретные производственные задачи. Агрономы нуждаются в инструменте, который возьмет на себя рутинную работу по планированию. ПО помогает разместить культуры по теплицам, учитывая их совместимость и правила чередования предшественников. Расчет потребности в посадочном материале также выполняется автоматически. Это освобождает время агронома для более важных решений.

Система предлагает оптимальные сроки посадки и своевременно предупреждает о возможных конфликтах в расписании. Все данные хранятся в единой базе, где собраны технологические карты, справочники культур и история предыдущих циклов. Такой подход позволяет работать с актуальной информацией и принимать обоснованные решения. Система учитывает доступные площади почв и временные окна, исключая накладки в графике.

С технической стороны предлагаемое решение проектируется как централизованное решение. Это означает, что все данные собираются в одном месте — на сервере с СУБД PostgreSQL. Такой подход упрощает поддержку актуальности информации и исключает конфликты данных: агроном в любой момент видит реальную картину по всем теплицам, а не разрозненные данные из разных источников. Для обмена информацией между частями системы используются стандартные протоколы, что обеспечивает надежность и совместимость с другим оборудованием.

Отдельное внимание уделено удобству работы. Интерфейс системы должен быть понятным даже тем пользователям, которые не имеют глубоких технических знаний. Все надписи, кнопки и сообщения об ошибках выводятся на русском языке. При проектировании учитываются требования доступности: интерфейс адаптирован для людей с особенностями зрения, поддерживает работу с программами экранного доступа. Используется светлая тема оформления, которая комфортна для глаз при длительной работе. Размер шрифтов подобран так, чтобы текст легко читался без напряжения.

Система должна работать круглосуточно, без выходных. Это важное требование, так как производственный процесс в теплицах не останавливается. Агрономы могут зайти в систему в любое время, чтобы проверить текущий статус или внести изменения в план. При этом ПО должно оставаться отзывчивым и быстрым даже при одновременной работе нескольких пользователей.

Важным аспектом является защита от ошибок. Если пользователь вводит некорректные данные или пытается выполнить невозможное действие, система не просто выдает сухое сообщение об ошибке, а поясняет, что именно пошло не так и как это исправить. Это снижает риск случайных ошибок и экономит время на поиск причин проблем.

2 ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ

2.1 Выбор инструментов или средств реализации проекта

2.1.1 Сводная таблица выбранных инструментов

В таблице 2.1 приложены все технологии, которые мы используем в нашем проекте, и их назначение.

Таблица 2.1 - сводная таблица используемых технологий

Технология	Назначение
oxlint, oxfmt	Линтинг и форматирование соответственно
Vitest	Фреймворк юнит-тестирования
react-testing-library	Вспомогательная библиотека для юнит-тестирования React-компонентов
Vite + @vitejs/plugin-react	Сборщик кода (bundler)
Storybook	Скриншотные тесты
Playwright	E2E тестирование
pnpm	Пакетный менеджер с упором на кеширование и поддержку монорепозитория
nx	Платформа управления монорепозиторием
react	Библиотека для поддержки JSX-компонентов
react-router	Маршрутизатор для SPA
react-admin	Библиотека готовых компонентов для административных систем
dhtmlx-gantt	Диаграмма Ганта
leaflet	Интерактивная карта местности
terra-draw + leaflet-adapter	Готовое решение для рисования и редактирования полигонов на карте местности. Интегрирован с leaflet через leaflet-adapter
TypeScript	Статический типизатор JS

PostgreSQL	Система управления базами данных (СУБД)
Docker + docker compose	Контейнеризация
Gitlab	Хостинг кода
Gitlab CI/CD	Пайплайн CI/CD
ExpressJS	Асинхронный маршрутизатор запросов
Zod	Валидация данных и поддержка строгих контрактов
Prisma ORM	Проектирование схемы БД, миграции, генерация клиентских типов
prisma-generator-plantuml-erd, prisma-generator-express	Генерация CRUD на ExpressJS и генерация ER-диаграммы
XiorJS	Абстракция над Fetch API. Отправка запросов и обработка ответов по HTTP/2.0
husky	git-хуки (CI/CD)
git	Система контроля версий (VCS)

2.1.1 Выбор СУБД

Мы выбрали СУБД PostgreSQL 18-ой версии. На то несколько причин: во-первых, наш проект требует поддержки геополитических данных формата geoJSON. PostgreSQL имеет плагин PostGIS, который предоставляет эту поддержку. Также этот плагин добавляет вспомогательные методы для поиска площадей полигонов — например, теплиц или самого предприятия, что исключает потребность самостоятельно реализовывать эту логику на бекенде.

Использование MongoDB нецелесообразно в контексте системы автоматизированного планирования культурооборота. Реляционная архитектура обеспечивает необходимую строгость связей, транзакционную надежность, эффективную обработку сложных запросов и прямую совместимость с математическими моделями оптимизации. Выбор реляционной СУБД соответствует отраслевым стандартам разработки модулей управления производством и информационных систем.

2.1.2 Выбор пакетного менеджера

Мы выбрали `pnpm`, потому что он заточен кеширование зависимостей и на работу с монорепопозиториями. `pnpm` не имеет таких возможностей.

Мы не рассматривали `yarn`, `bun` и `deno`, так как они не имеют такой гибкой поддержки монорепопозитория, как `pnpm`. Также изучение и интеграция данных инструментов требует времени.

2.1.3 Назначение библиотек тестирования

Как видно в таблице 2.1, мы выбрали сразу несколько инструментов для тестирования. Мы решили так сделать, так как это стандарт индустрии — данные инструменты покрывают свои ниши тестирования, и использования только части из них не достаточно для полного покрытия.

`Storybook` необходим для проверки внешнего вида интерфейса на разных платформах.

`Vitest` позволяет писать `unit`-тесты на фронтенде и бекенде, к тому же он очень быстрый и интегрирован с `vite`. Но одного `vitest` не достаточно - для тестирования `React`-компонентов целесообразно использовать соответствующую вспомогательную библиотеку, с чем справляется `react-testing-library`. Запускается он в среде `jsdom` (а не в `browser`), так как работа с браузером покрыта `E2E`-тестами, а лишняя интеграция с эмуляцией окружающей среды браузера привносит свои краевые случаи и полностью лишает нас скорости `vitest`.

`Playwright` также интегрирован в экосистему `vite`, и является безоговорочным лидером в `E2E`-тестировании.

2.1.4 Выбор ORM-системы

В таблице 2.2 мы рассматривали несколько ORM-систем, включая Prisma, TypeORM, Drizzle, MikroORM и Sequelize.

Таблица 2.2 - сравнительная таблица популярных ORM-решений

ORM	Поддержка плагинов	Типизация	Транзакционная согласованность	Валидация	Производительность	Управление миграциями	Необходимость в драйвере
Prisma ORM	Да	Полная	Хорошая	Полная	Низкая	Декларативное	Нет
TypeORM	Да	Полная	Полная	Хорошая	Низкая	Императивное	Да
Drizzle ORM	Ограниченная	Полная	Полная	Полная	Высокая	Декларативное	Да
MikroORM	Да	Полная	Полная	Хорошая	Средняя	Императивное	Да
Sequelize	Да	Очень слабая	Полная	Хорошая	Низкая	Императивное	Да

Как видно, Prisma имеет несколько преимуществ перед другими ORM. Рассмотрим некоторые особенности Prisma подробнее.

Во-первых, схема БД прописывается в одном или нескольких файлах с расширением `.prisma`. Prisma имеет декларативный синтаксис, который напоминает SQL. Хотя этот синтаксис и ограничен и не поддерживает триггеры и проверку входных данных (помимо NOT NULL, UNIQUE), но это можно нивелировать на стороне бекенда.

Также Prisma изначально поддерживает миграции (система Prisma migrations) и генерацию клиентских типов (`@prisma/client`).

Prisma поддерживает пользовательские генераторы. Мы используем `prisma-generator-express` — генератор CRUD на ExpressJS для каждой сущности в

схеме, и `prisma-generator-plantuml-erd` — генератор ER-диаграммы в форматах PlantUML, Markdown и txt.

2.1.5 Выбор хостинга кода

Мы выбрали Gitlab, а не Github, потому что наш проект имеет Gitlab CI/CD пайплайн. Этот пайплайн поддерживает только Gitlab.

2.1.6 Выбор HTTP-обработчика

Мы выбрали Xior по нескольким причинам:

- Xior предоставляет интерфейс, схожий с Axios, что удобно;
- мы не доверяем axios из-за его недавнего взлома¹ (31 марта 2026 года). Мы не хотим подвергать себя опасности, зная, что есть альтернативы;
- мы хотим использовать fetch API, но не хотим изобретать велосипед;
- мы хотим поддержку плагинов, при необходимости какой-то функциональности, которой нет в Xior по умолчанию. Мы не хотим реализовывать эту функциональность самостоятельно.

2.2 Разработка проекта решения

2.2.1 Упрощенная система ролей

Функции формирования нормативной базы (роль эксперта), оперативного планирования циклов (роль агронома), создания физического предприятия и расстановки теплиц (роль директора) будут консолидированы в едином пользовательском интерфейсе агронома. Роли производственного персонала не будут реализованы вовсе.

¹ <https://www.stepsecurity.io/blog/axios-compromised-on-npm-malicious-versions-drop-remote-access-trojan>

Данные решения обусловлены задачами курсового проектирования: демонстрацией алгоритмов оптимизации культурооборота без усложнения системы модулями ролевого согласования. Схема базы данных учитывает наличие ролей, не реализуемых в рамках MVP, потому систему возможно масштабировать для точного отражения реального положения вещей.

2.2.1 Диаграмма сценариев использования

На диаграмме сценариев использования (смотрите рис. 2.2.1.1) отображены три актора системы: Гость, Агроном и Администратор.

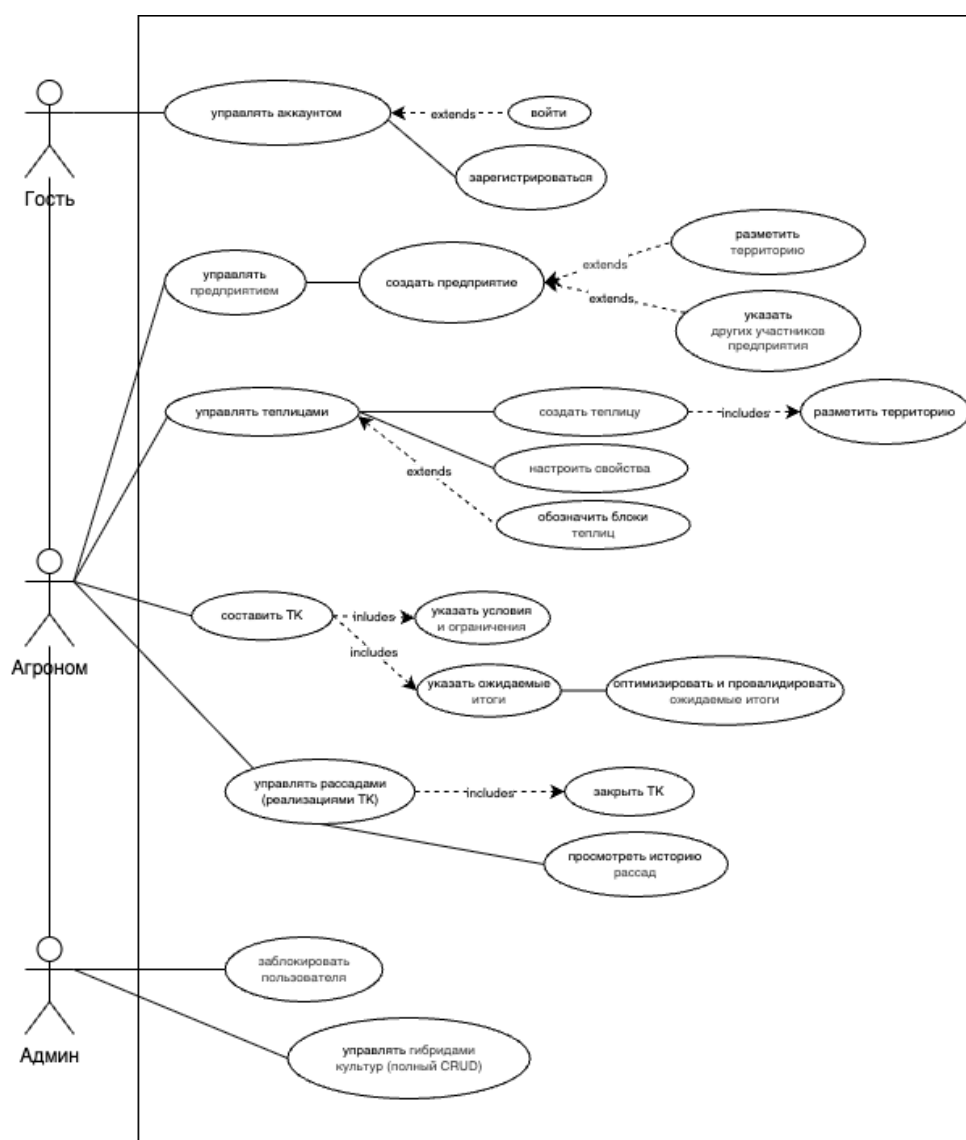


Рисунок 2.2.1.1 — диаграмма сценариев использования

Гость имеет возможность управлять своим аккаунтом, что включает в себя функции входа в систему, если аккаунт существует, и регистрации аккаунта.

Агроном как ключевой пользователь системы обладает наиболее широким функционалом. Он может управлять предприятием, что подразумевает создание нового предприятия с возможностью разметки территории и указания других участников предприятия. Агроном управляет теплицами: создает новые теплицы с обязательной разметкой территории, настраивает их свойства и обозначает блоки теплиц. Важной функцией является составление технологической карты, которое включает указание условий и ограничений, а также определение ожидаемых итогов с их последующей оптимизацией и валидацией. Агроном управляет рассадами, то есть реализациями технологических карт, что включает закрытие технологической карты и просмотр истории рассад.

Администратор системы выполняет функции модерации и управления справочными данными. Он может блокировать пользователей при необходимости, а также управлять гибридами культур, выполняя полный цикл операций CRUD: создание, чтение, обновление и удаление записей о гибридах.

2.2.2 Entity-Relationship диаграмма (ERD)

Ниже на рисунке 2.2.2.2 представлена ER-диаграмма базы данных, которая отражает структуру хранения информации в разрабатываемой системе. Схема описывает взаимосвязи между основными сущностями предметной области и обеспечивает целостность данных на уровне СУБД.

Инфраструктура теплиц описывается таблицей GreenhouseBlock, которая принадлежит предприятию и ссылается на тип почвы в справочнике Soil. Конкретные теплицы хранятся в таблице Greenhouse и связаны с блоками, а также со справочниками форм GreenhouseShape и материалов GreenhouseMaterial через отношения многие-к-одному. Справочник CultureSoilRecommendation фиксирует совместимость культур и типов почв, связывая таблицы Crop и Soil.

Справочная информация о культурах и сортах организована иерархически: таблица Crop содержит виды культур, а Variety хранит конкретные гибриды, связанные с культурой через отношение один-ко-многим. Каждый сорт привязан к предприятию, что позволяет вести учет используемых гибридов в рамках конкретного хозяйства.

Представленная схема базы данных соответствует третьей нормальной форме (3НФ). Все неключевые атрибуты в каждой таблице зависят исключительно от первичного ключа и не содержат транзитивных зависимостей. Например, в таблице TechnologicalCard параметры вроде plantingDensity или growthDurationDays описывают непосредственно технологическую карту, а не какие-либо другие сущности. Справочные данные вынесены в отдельные таблицы: культуры, сорта, типы почв, формы и материалы теплиц хранятся независимо и подключаются через внешние ключи, что исключает избыточность и аномалии при изменениях в записях. В схеме присутствуют ассоциативные таблицы, вроде UserRoleAssignment или PlantingGreenhouseBlock, которые содержат только внешние ключи для реализации связей многие-ко-многим. Без них схема бы не соответствовала 3НФ.

2.2.3 Диаграмма последовательности

Рисунок 2.2.2.1 — диаграмма последовательности

3 ПРАКТИЧЕСКАЯ ЧАСТЬ

3.1 Разработка составных элементов

3.2 Установка и настройка

3.3 Тестирование

3.4 Ввод в эксплуатацию

3.5 Разработка сопроводительной документации

Заключение

Список использованных источников

- <https://radiotochki.net/blog/seedlings/rassadnyy-metod-kak-obmanut-prirodu-i-poluchit-ranniy-urozhay-dazhe-v-usloviyah-korotkogo-leta.html>
- <https://exactfarming.com/tekhkarta>
- <https://www.stepsecurity.io/blog/axios-compromised-on-npm-malicious-versions-drop-remote-access-trojan>

Приложения

Приложение А: техническое задание продукта

А.1 Общие сведения

Полное наименование проекта: Аграрио - система контроля культурооборота на ПОТК

Краткое наименование проекта: Аграрио

Домен: agrar.io

А.2 Назначение и цели создания системы

А.2.1 Назначение системы

Система предназначена для автоматизации планирования реализации технических карт на основе наличия необходимых площадей теплиц, требований к урожайности конкретных гибридов и срокам реализации ТК, а также учет предшественников для исключения развития патогенов.

В рамках проекта автоматизируется информационно-аналитическая деятельность в следующих бизнес-процессах:

- нанесение карты местности предприятия с валидацией, например, исключением постановки вершин предприятия непосредственно на водоемах;
- нанесение теплиц на карту местности предприятия с исключением выхода вершин теплицы за границы предприятия и исключением постановки вершин непосредственно на водоемах;

- объединение отдельных теплиц в блоки (группы) теплиц с возможностью редактирования и удаления отдельных блоков без нарушения исторических сведений о реализованных ТК;
- подбор одной или более ТК из банка ТК данного предприятия;
- указание требований к реализации подобранных ТК и сопоставление этих требований с реальными возможностями производства, немедленное оповещение о невозможности реализации указанных требований или о возможности их оптимизации;
- поиск наиболее оптимального способа задействовать незанятые площади теплиц и блоков теплиц с учетом всех ключевых переменных, включая предшественников и непосредственных требований по ТК;
- визуализация реализуемых и уже реализованных ТК в виде временной шкалы (диаграммы Ганта) с возможностью непосредственного редактирования сроков реализации ТК с проверкой при помощи вышеуказанного алгоритма оптимизации;
- хранение и чтение исторических сведений о реализованных ТК на временной шкале, использование этих сведений для учета предшественников.

А.2.2 Цели создания системы

Система создается с целью:

- автоматизации планирования и оптимизации культурооборота в сооружениях защищенного грунта с учетом агротехнических и ресурсных ограничений;
- повышения эффективности использования производственных площадей за счет математического алгоритма распределения циклов и минимизации простоев теплиц;
- обеспечения соблюдения фитосанитарных норм и правил чередования культур-предшественников для снижения рисков заболеваний;
- формирования единого информационного пространства для согласованной работы специалистов разного профиля и разграничения прав доступа;

- повышения качества, точности и своевременности производственного планирования за счет исключения ручных расчетов и человеческого фактора;
- предоставления инструментов визуализации и контроля выполнения графика производственных циклов в реальном времени.

А.3 Требования к системе

Система должна быть централизованной, т.е. все данные должны располагаться в центральном хранилище.

В качестве протокола взаимодействия между компонентами системы на транспортно-сетевом уровне необходимо использовать протокол TCP/IP.

Для организации информационного обмена между компонентами системы должны использоваться специальные протоколы прикладного уровня, такие как: HTTP или его расширение HTTPS, PostgreSQL wire protocol.

Для организации доступа пользователей к отчетности должен использоваться протокол презентационного уровня HTTP или его расширение HTTPS.

А.3.1 Требования к источникам данных системы

Источниками данных для системы должны быть:

- информационная система управления предприятием, содержащая данные о производственных циклах, посадках и привязке к блокам теплиц (СУБД PostgreSQL версии 17 или выше);
- информационно-справочная система, включающая нормативные параметры, технологические карты, справочники культур и гибридов (СУБД PostgreSQL версии 17 или выше);
- система управления организационной структурой, хранящая учетные записи пользователей, роли и права доступа (СУБД PostgreSQL версии 17 или выше);

- система учета инфраструктуры, содержащая реестр теплиц, блоков, их характеристик и параметров почв (СУБД PostgreSQL версии 17 или выше);
- система аналитики и отчетности, агрегирующая исторические данные циклов для формирования сводных показателей и диаграмм (СУБД PostgreSQL версии 17 или выше).

Взаимодействие с внешними информационными системами, включая сторонние API и БД, на текущем этапе разработки не предусмотрено. Система функционирует как автономное решение с внутренней архитектурой «клиент-сервер».

А.3.2 Требования к режимам функционирования системы

Система должна поддерживать следующие режимы функционирования:

- основной режим, в котором подсистемы выполняют все свои основные функции.

В основном режиме функционирования система должна обеспечивать;

- работу пользователей в режиме – 24 часов в день, 7 дней в неделю (24x7);
- выполнение своих функций – сбор, обработка и загрузка данных; хранение данных, предоставление отчетности.

А.3.3 Требования к эргономике и технической эстетике

Подсистема формирования и визуализации отчетности данных должна обеспечивать удобный для конечного пользователя интерфейс, отвечающий следующим требованиям.

В части внешнего оформления:

- должно быть обеспечено наличие локализованного (русскоязычного) интерфейса пользователя;

- должны соблюдаться требования доступности интерфейса для людей с особенностями зрения, включая различные формы дальтонизма, слабовидящих и слепых людей;
- интерфейс должен быть адаптирован для программ экранного доступа (скринридеров) в соответствии с рекомендациями и требованиями WCAG 2.1 (Web Consortium Accessibility Guidelines);
- должна присутствовать комфортная для глаза светлая тема интерфейса;
- размер шрифта обычного текста должен быть минимум 14px (пикселей), заголовков — минимум 12px;
- используемые шрифты должны быть удобными для продолжительного чтения.

В части диалога с пользователем:

- при возникновении ошибок в работе подсистемы или в вводимых пользователем данных на экран монитора должно выводиться сообщение с наименованием ошибки, по возможности с рекомендациями по её устранению на русском языке;